

```
marp: true
theme: default
paginate: true
style: |
section {
font-family: 'Inter', 'Helvetica Neue', sans-serif;
font-size: 22px;
color: #1a1a2e;
background: #ffffff;
}
h1 { font-size: 2em; color: #0f3460; border-bottom: 3px solid #e94560; padding-bottom: 12px; }
h2 { font-size: 1.5em; color: #16213e; }
code { background: #f8f9fa; padding: 2px 6px; border-radius: 4px; font-size: 0.9em; }
table { font-size: 0.85em; }
.highlight { background: #fff3cd; padding: 4px 8px; border-radius: 4px; }
```

■ Redrob Candidate Ranker

Read the Story. Not the Keywords.

Hackathon: Intelligent Candidate Discovery & Ranking

Role: Senior AI Engineer — Founding Team, Series A

The Problem: Why Keyword Filters Fail

The baseline submission (sample_submission.csv) ranks:

#1	CAND_0004989	**HR Manager**	9 AI core skills
#2	CAND_0001195	**HR Manager**	9 AI core skills
#3	CAND_0003114	ML Engineer	4 AI core skills
#4	CAND_0000339	**Content Writer**	8 AI core skills

“An HR Manager ranking above an ML Engineer. This is what happens when you trust `skills[]` instead of `career_history[].description`.”

The dataset is **adversarially designed** to punish keyword matching. We built a system a great recruiter would trust instead.

Data Findings (Verified on 100K Candidates)

Key insight: **~133 unique skill names appear on nearly everyone** (including accountants).

The signal is in *descriptions*, not *skill lists*.

The Trap We Refused To Fall For

Why skill-list counting fails:

The honeypot threat:

“Search Engineer”, headline “ML, NLP, Recommendation Systems”,

“loaded with Pinecone/FAISS — but `years\_of\_experience=2.7`”

*“while career history totals **18 months** (impossible).”*

Caught purely by **internal numerical contradiction**, never by keywords.

What the JD Actually Requires

We distilled the JD into `rubric.yaml` — produced **once**, **offline**:

Production embeddings/retrieval	25%	career descriptions
Vector DB / hybrid search infra	20%	career descriptions
Shipped ranking/recsys system	25%	career descriptions
Eval rigor (NDCG/MRR/A-B)	15%	career descriptions
Strong Python engineering	15%	career descriptions

Experience band: Soft peak at 5–8 yrs, gentle penalty outside, never a hard filter.

Location: Noida/Pune preferred; Hyderabad/Mumbai/Delhi/Bangalore welcome; international = heavy down-weight (no visa sponsorship).

System Architecture

Feature Engineering: What We Measure

All 8 base features are **interpretable** and can be explained in an interview:

`role_fit_dense`	Semantic fit to JD anchors	Cosine(candidate_emb, anchor_embs).max()
`role_fit_sparse`	Keyword fit in descriptions	BM25 on career text
`evidence_eval`	NDCG/A-B/offline-online language	Regex on descriptions
`product_ratio`	Months at product-AI vs services	Company classification
`yoe_band`	Smooth Gaussian peak at 5–8 yrs	Piecewise linear
`tenure_stability`	No title-chasing	Short stints ratio
`recent_handon`	Still writing code	Code signals in latest role
`location_fit`	Noida/Pune→1.0, intl→0.4	Lookup table

Multiplicative Penalties: Defense in Depth

Why multiplicative? Keeps the ranking smooth and defensible.

No hard filters → no brittle cutoffs → graceful degradation.

Honeypot Defense: Internal Consistency

The 4 contradiction axes:

Result: 15 near-honeypots detected (score < 0.15), 0 in our top-100.

No hardcoded IDs — structural detection only.

Behavioral Availability: The Hireability Multiplier

"A perfect-on-paper candidate who hasn't logged in for 6 months"

"and replies to 5% of recruiters is not actually hireable."

Effect example:

- ML Engineer, perfect role fit, stale (180d inactive, 5% response) → multiplier 0.63
- Same candidate, active this week, 80% response → multiplier 0.97

Range: [0.60, 1.00] — never zeros out a perfect fit, but discounts strongly.

Evaluation: Pseudo-Label Validation & Ablations

Pseudo-labels (0–5 tiers) generated from rubric without ground truth:

5	India + product-AI + retrieval in desc + eval rigor + 5–9 yrs + active	61
4	India + product-AI + retrieval + 4–11 yrs + active	219
3	India + retrieval in desc + reasonable YOE	135
2	Tech role, embedding/vecdb mentions, no ranking	12
1	Edge cases, international with relocate	22,155
0	Honeypots, stuffers, non-tech, intl/no-relocate	77,418

Ablation results (pseudo-NDCG on 100K, measured):

Dense-only	0.4488	0.3821	0	0
Hybrid (no signals)	0.6522	0.6069	0	0
Hybrid + signals	0.7031	0.6976	0	0
Full composite	**0.8508**	**0.8603**	0	0

Top-10 stability: 100% — all top-10 stay top-10 across 20 trials of $\pm 15\%$ weight perturbation.

Why Hybrid? The Design Rationale

Dense embeddings (BGE-small-en-v1.5):

- Captures semantic similarity to JD facets (not keyword matching)
- Clusters templated descriptions together → good for pooling
- BUT: templated descriptions mean all "Search Engineers" look alike at the top
- Can't differentiate among the ~3,800 strong candidates alone

Sparse / BM25:

- Rewards exact technical terminology (NDCG, Pinecone, LambdaMART)
- Fast — vectorized score over 100K docs in <2s
- Over-rewards keyword stuffers if used alone

Why no per-candidate LLM calls:

- Compute constraint: ≤5 min, CPU only, no network
- 100K × LLM call = impossible
- LLMs add hallucination risk to reasoning; we use grounded rules instead
- The dense encoder is an LLM under the hood — we use it correctly (batch, offline)

Reasoning Generation: Grounded and Varied

Each top-100 candidate gets a 1–2 sentence reasoning built from their **actual fields**:

Rank 1 (ideal fit):

""Senior Machine Learning Engineer (6.1 yrs) at Genpact AI: built and shipped a production recommendation system at a marketplace product, going from offline ex. Pune-based, open to relocate; 88% recruiter response, active this month, open to work.""

Rank 4 (strong fit):

""Recommendation Systems Engineer (6.0 yrs) at Sarvam AI: led the migration from keyword-based to embedding-based search across a 30m+ candidate corpus. Bangalore-based; 79% recruiter response, active recently.""

Rank 90+ (filler, honest):

""Backend Engineer (4.3 yrs) at Infosys — adjacent pipeline work, no retrieval evidence. Hyderabad-based; 45% response rate; concern: entire career in services firms.""

Rules enforced: No hallucinated skills/employers, tone matches rank, concerns surfaced honestly.

Compute Story

Precompute	Parse 100K + embed (BGE-small, MPS)	~30 min (run once)
Precompute	BM25 index build	~103s
Rank step	Load artifacts	~2.5s
Rank step	Parse 100K JSONL	~78s
Rank step	Consistency + availability	~2s
Rank step	Feature engineering (vectorized)	~1.5s
Rank step	LTR score + penalties	~1.5s
Rank step	Top-100 select + reasoning	~0.1s
Total rank step	**86 seconds**	✓ (214s to spare)

Memory: ~1.6 GB (embeddings 146 MB + BM25 198 MB + DataFrame)

Disk: ~146 MB embeddings, ~198 MB BM25, ~2.6 MB features parquet

Limitations & Next Steps

Current limitations:

- Pseudo-labels are rubric-derived, not recruiter-labeled
- Description templating limits discrimination among top-tier candidates

→ Rely on structured signals (company, YOE, location, availability) to differentiate

- Consistency detector calibrated without ground truth for the ~80 honeypots

With real recruiter feedback:

1. **Online LTR:** True LambdaMART on recruiter click/accept labels → replaces pseudo-labels
2. **A/B testing infrastructure:** Track which ranking leads to interviews/hires
3. **Pairwise labels:** "Was candidate A better than B for this JD?" → more signal
4. **Cross-JD generalization:** Train one ranker per JD family, not per JD

The system is deliberately transparent — every number is defensible because every feature is named, weighted, and interpretable.

Summary

`parse.py`	JSONL → DataFrame; descriptions weighted 3× over skills
`consistency.py`	4-axis numerical contradiction detector (honeypot guard)
`precompute.py`	BGE-small embeddings + BM25 index + JD anchors
`features.py`	8 interpretable features, fully vectorized
`signals.py`	Behavioral availability multiplier [0.60, 1.00]
`score.py`	Composite × penalties (6 multiplicative negative factors)
`ltr.py`	LightGBM LambdaMART on pseudo-labels (calibration)
`reason.py`	Grounded, varied, concern-aware reasoning per candidate
`rank.py`	End-to-end endpoint; <90s on CPU; passes validator

One command to reproduce:

"""Read the story, not the keywords."*"*